# Stateless Session Beans

Chapter 5

# Introduction

- Persistence beans encapsulate the business objects in a system
- They make it "easy" to share business objects across applications
- Session beans are used to describe business processes and task flow
  - Manage the interactions between persistence beans
  - Process the business logic that does not easily fit in a single persistence bean
  - Provide interfaces to the outside world for persistence beans
  - Use entities for persistence or to go directly to the database

# Three Session Beans

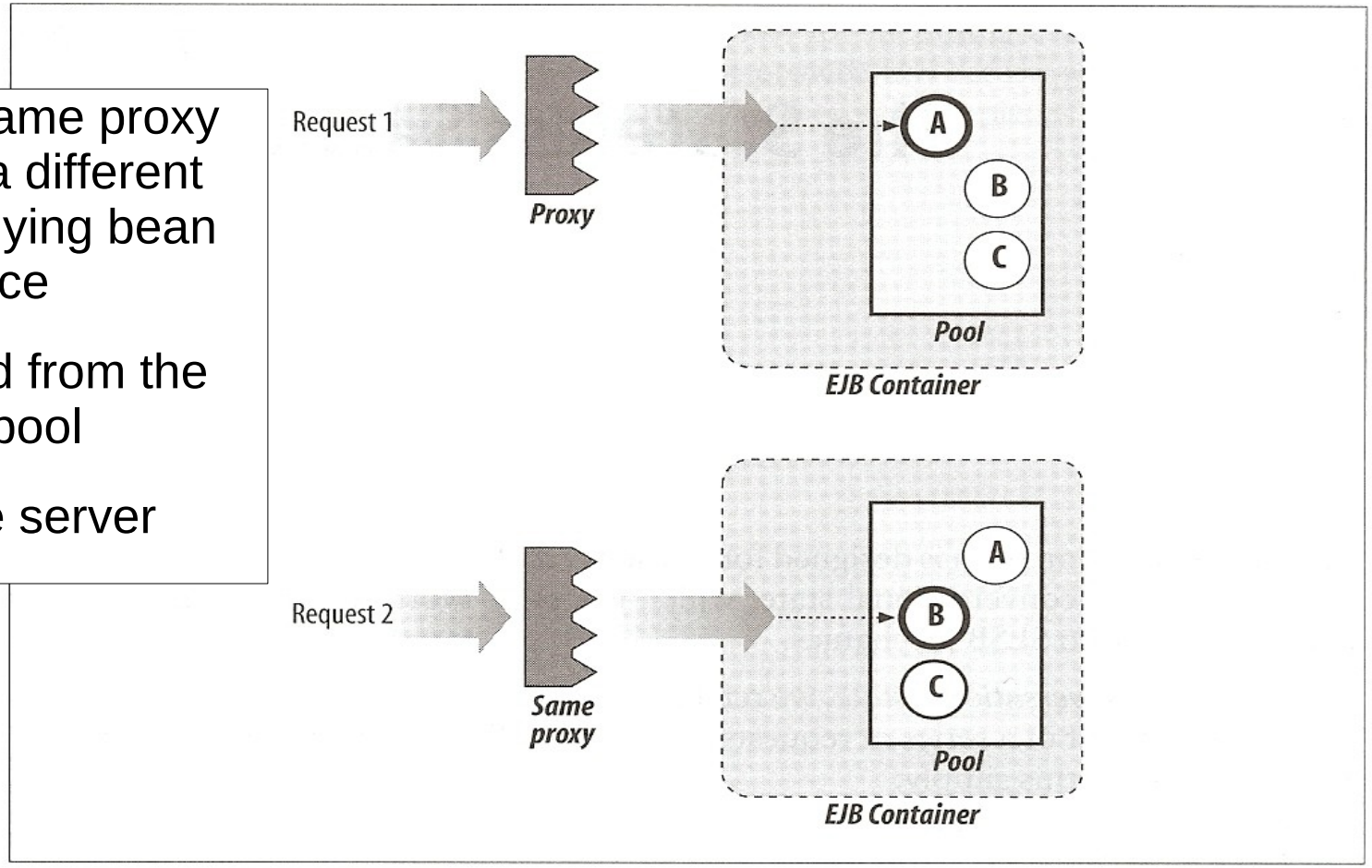
- Stateless, stateful and singleton
  - Short-lived, not persistent
- *Stateless* beans: collection of related services
  - Each service represented by a method
  - Each method invocation is independent of others
- *Stateful* beans hold client information between method calls
  - Represent logic that could have been stored in client application
  - Client holds bean instance which is not shared
  - Server may have timeout period to limit storage waste
- *Singleton* beans backed by single instance
  - All business requests serviced by same backing object

# Stateless Session Bean (SLSB)

- Very efficient and easy to develop
- Bean instances shared between multiple clients
  - Because no conversational state maintained
- Does not require passivation or activation
- Each method should be self-contained
  - Completes an entire business task
  - Can run in a transaction

# Session Bean Pooling

- The same proxy uses a different underlying bean instance
- Picked from the bean pool
- By the server



# SLSB Restrictions

- Stores no client-specific information between method calls
- Can think of a method call as equivalent to a transaction
- Can store information from SessionContext and JNDI Enterprise Naming Context (ENC)
- Can have instance variables
  - Anything stored should be independent of client
  - No guarantee client will use same bean instance for successive method calls

# XML Deployment Descriptor

- EJB has optional deployment descriptor
  - In META-INF/ejb-jar.xml file of EJB's JAR file
- Can use as alternative to annotation
  - Or for metadata which is not available in annotations
- <enterprise-beans> element in <ejb-jar> element defines set of EJBs being deployed
  - <session> element defines a session bean
    - <ejb-name> gives the bean a name
    - <remote> and <local> define business interfaces
    - <ejb-class> identifies bean class
    - <session-type> indicates Stateful/Stateless/Singleton

# jakarta.ejb.SessionContext

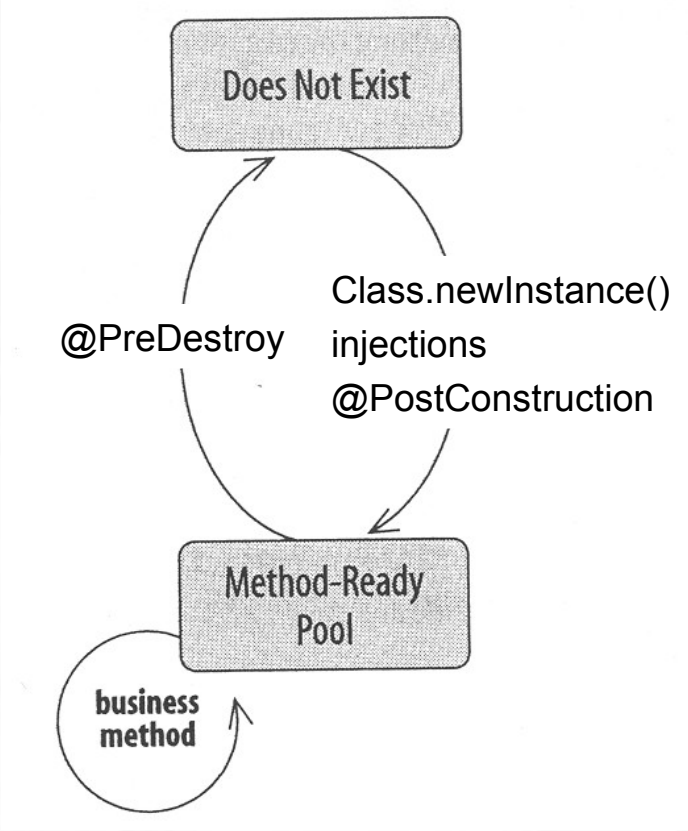
- This is instance's interface to EJB container
- Session bean can use @Resource to get reference
  - @Resource SessionContext ctx;
- Session bean can get reference to *its* EJB reference if needed to pass to other beans
  - <T> getBusinessObject(Class<T> businessInterface)
    - argument is business local or remote interface
  - Class getInvokedBusinessInterface
    - returns local, remote or web service interface bean was called through
  - Illegal for bean instance to pass **this** reference to another bean

# jakarta.ejb.EJBContext

- SessionContext extends EJBContext
- Allows bean class to get
  - timerService (covered in Ch. 19)
  - callerPrincipal and isCallerInRole
    - represent client calling bean
    - principal.getName() is typically login name
  - Transaction information (covered in Ch. 17)
    - userTransaction, rollbackOnly (get and set)
- lookup() method allows looking up entries in EJB's ENC (see below)

# Stateless Session Bean Life Cycle

- Only two states
  - Does  not exist
  - method-ready pool
- Efficient to pool beans
  - ENC setup only required once



# Method-Ready Pool

- SLSB instances enter Method-Ready Pool as container needs them
  - May create pool of instances in advance depending on EJB Container parameters
- When a new instance is created:
  - bean is instantiated with Class.newInstance()
    - requires no-argument constructor
  - resources are injected per annotations/XML
  - Then PostConstruct event occurs
  - SLSB can keep needed resources open for life of instance
  - Close resources when PreDestroy event occurs

# Method-Ready Pool Behaviour

- When client calls SLSB business method, any available instance in Method-Ready Pool can process it
  - Instance is unavailable until method completes then goes back to pool
  - While in use, its SessionContext reflects the current EJBObject that fronts for the object
    - Instance can determine transaction and security information relevant to client call
- When client not in middle of method call, bean is freed for use by other clients

# Method-Ready Pool Behaviour

- Clients get a reference to a stateless session bean by having reference injected or looking it up with JNDI
  - Bean's @PostCreate method is not called
    - Was called when bean instance was created by container
  - Injection/lookup creates an EJBObject, not the bean instance!
- Similarly, when the client removes instance this removes the EJBObject, not the bean instance
  - Container decides when to delete bean instance and call @PreDestroy method

# Example: EncryptionEJB

- Frequently users authenticated with userName/password
- Sometimes need to store sensitive information
  - Such as credit card number
- Storing the data in plain text allows compromise by accessing the database
- Good practice is to encrypt the information
  - Or use one-way hash if do not need to retrieve the information
- Example handles these functions

# Suitability of SLSB

- Application uses simple request-response format
  - All of the required information is in the request
    - Passed with method parameters
  - The result does not depend on saved client information
    - Each request is independent of others
- Application handles hashing and two-way encryption

# Business Interface

```
public interface EncryptionCommonBusiness {
   String encrypt(String input) throws IllegalArgumentException, EncryptionException;
   String decrypt(String input) throws IllegalArgumentException, EncryptionException;

   String hash(String input) throws IllegalArgumentException, EncryptionException;

   Future<String> hashAsync(String input) throws IllegalArgumentException, EncryptionException;

   boolean compare(String hash, String input) throws IllegalArgumentException, EncryptionException;

   /*
    * This comment applies to all below this marker.
    *
    * In real life it's a security risk to expose these internals,
    * but they're in place here for testing and to show
    * functionality described by the examples.
    */

   String getCiphersPassphrase();
   String getMessageDigestAlgorithm();
}
```

# Local and Remote Interfaces

- As earlier, example defines both of these
  - `public interface EncryptionLocalBusiness extends EncryptionCommonBusiness`
  - `public interface EncryptionRemoteBusiness extends EncryptionCommonBusiness`
- A business interface can be local *or* remote
  - Not both
- Remote interface can be invoked from networked clients
  - Parameters and return values passed by copy (call-by-value)
- Local interface cam be invoked from same local JVM
  - Parameters and return values passed by reference (call-by-reference)

# Exception Handling

- Example throws `EncryptionException`
- The bean developer is responsible for using the right exceptions
  - Runtime Exceptions are System Exceptions
  - Other System Exceptions
  - Application exceptions
- Following slides discuss these

# System Exceptions

- `java.lang.RuntimeException`
- Indicate serious programming errors
  - Unchecked, do not need to declare or catch
- Bean should not usually catch these
  - Let container handle them
  - Container will wrap in `EJBException`
- Bean can also throw `EJBException` for serious problems
- Jakarta Persistence uses runtime exceptions
  - Consider changing to application exceptions
  - Add `@jakarta.ejb.ApplicationException`

# Application Exceptions

- EJB methods can throw application exceptions
  - Passed back to caller
- Indicate business logic error
  - Not programming error
  - Here, problem performing encryption: EncryptionException
- Consider appropriate exception type
  - EJBException for detected system problems
  - Application exceptions for business logic issues
  - Sometimes not obvious
    - EntityNotFoundException: system or logic error?
  - Transaction implications covered in Chapter 17

# Exceptions from Other Subsystems

- Think how to handle these
  - such as `SQLException` from JDBC call
- Usually wrap in `EJBException` and rethrow
  - Otherwise container will pass them back to client
  - Interface will need to declare them
- Note `EJBException` is an unchecked exception
  - Avoids having to declare or catch these
- If you know the cause
  - Wrap as an appropriate application exception

# Bean Class (EncryptionBean)

- Most of the work here (see code)
- Annotations on the bean implementation class declare SLSB and business interfaces
  - Usually annotate business interfaces with @Local or @Remote
- Name declares EJB Name
  - Defaults to lower-cased class name

```
@Stateless(name = EncryptionBean.EJB_NAME)
@Local(EncryptionLocalBusiness.class)
@Remote(EncryptionRemoteBusiness.class)
public class EncryptionBean implements
     EncryptionLocalBusiness, EncryptionRemoteBusiness
```

# Accessing Environment Properties

- Bean implementation declares instance fields as part of internal state
  - OK, since not visible to client
- Ones below are configurable attributes initialized via *environment entries* (in ENC)

```
/** loaded via SessionContext.lookup */
private String ciphersPassphrase;

/**injected via @Resource annotation with name
  * property equal to env-entry name */
@Resource(name =
         ENV_ENTRY_NAME_MESSAGE_DIGEST_ALGORITHM)
private String messageDigestAlgorithm;
```

# JNDI Enterprise Naming Context (ENC)

- Each EJB has its own JNDI namespace
  - Called the *Enterprise Naming Context* (ENC)
  - Referenced by "java:comp/env/..." in JNDI lookup
  - Or injected using @Resource
- Can hold configuration information for the EJB
  - Defined in EJB's XML deployment descriptor
  - Intended to be defined by person that deploys application
- @Resource and @EJB annotations are easy ways of accessing objects registered in ENC
- ENC described in more depth in Chapter 16

# ejb-jar.xml

```
<ejb-jar xmlns="http://jakarta.ee/xml/ns/jakartaee"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://jakarta.ee/xml/ns/jakartaee
                  http://jakarta.ee/xml/ns/jakartaee/ejb-jar_4_0.xsd"
  version="4.0">
  <enterprise-beans>
    <session>
      <ejb-name>EncryptionEJB</ejb-name>
      <!-- Override the ciphers' default  passphrase -->
      <env-entry>
        <env-entry-name>ciphersPassphrase</env-entry-name>
        <env-entry-type>java.lang.String</env-entry-type>
        <env-entry-value>OverriddenPassword</env-entry-value>
      </env-entry>
      <!-- Override the default unidirectional hash MessageDigest algorithm -->
      <env-entry>
        <env-entry-name>messageDigestAlgorithm</env-entry-name>
        <env-entry-type>java.lang.String</env-entry-type>
        <env-entry-value>SHA</env-entry-value>
      </env-entry>
    </session>
  </enterprise-beans>
</ejb-jar>
```

# Asynchronous Methods

- New in EJB 3.1
  - Available on any session bean business method
- Asynchronous method returns immediately
  - EJB method proceeds in parallel
- If method returns a value, use java.util.concurrent.Future<E> as return type
  - Java class that holds value of type E
    - Value of type E available in the future
    - Retrieve value using Future.get()
      - Blocks until value available
- Annotate method with jakarta.ejb.Asynchronous

# Summary

- Stateless session beans are conceptually easier than stateful beans
  - Each method should accomplish a complete task for the caller
    - Rather than one step in a task which requires storing customer state
  - Each method should be independent
- Stateless is also more efficient of memory
  - Can be pooled
- EJBs can be configured using their Enterprise Naming Context